



Polishing Monitoring & Alerting Dashboards

From raw metrics to **actionable insights**

Creating observability that serves engineers, SREs, ML teams, and stakeholders with clear, reliable, production-ready monitoring

Why Dashboard Polish Matters

Raw dashboards lead to **signal overload** - engineers can't identify critical issues quickly

Stakeholders need **simple, business-aligned KPIs** that translate technical metrics into business impact

Professional observability builds credibility in your Capstone defense and demonstrates production readiness



- 📄 Polished dashboards ensure runbooks link directly from alerts, creating a seamless troubleshooting experience

Core Dashboard Families

1

API & Inference Health

Latency, error %, QPS

2

Model Quality

Drift, accuracy proxies, safety metrics

3

Infra/Cluster Health

GPU, CPU, memory, pod restarts

4

Data Pipeline Freshness

Feature staleness, Kafka lag

5

Cost & Efficiency

GPU hours, \$/1k requests, idle %

6

SLO Compliance

Burn rate, uptime %

Dashboard Hygiene Rules

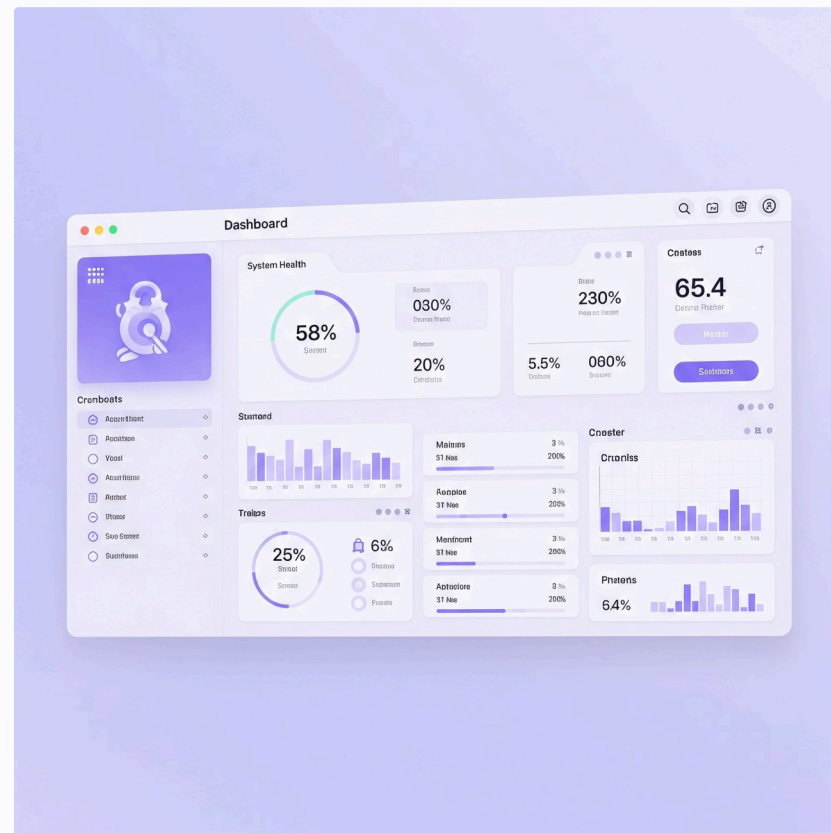
Keep it focused: Limit to ≤ 12 panels per dashboard

Group by persona: SRE, ML engineer, product views

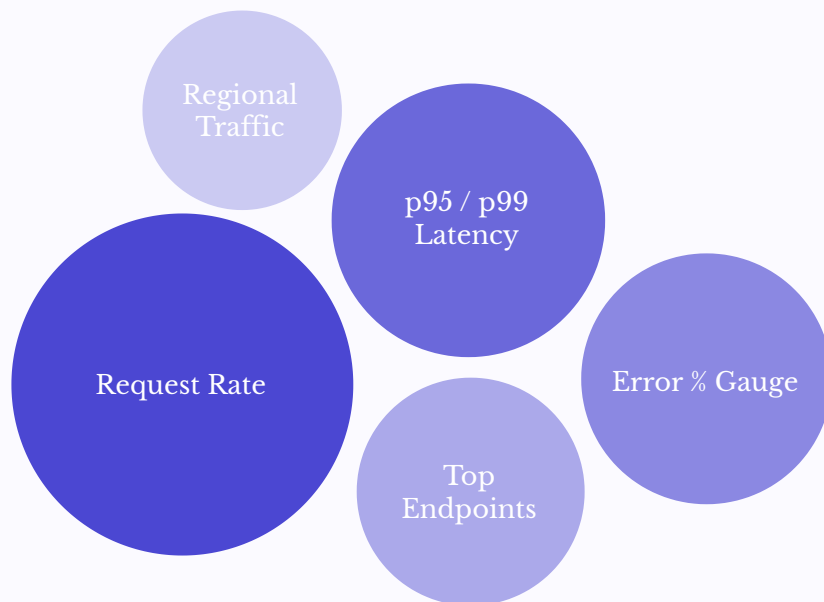
Be consistent: Standardize units, thresholds, colors

Prioritize SLOs: Place RAG status tiles first for at-a-glance health

Track changes: Always show model version + release markers



Inference Dashboard Example



Key Panels

- Request rate, p95/p99 latency, error %
- Top endpoints by latency
- Regional breakdown (map or table)
- HPA scaling vs traffic
- Cost per 1k requests

PromQL Example

```
histogram_quantile(0.95,  
  sum by (le) (  
    rate(http_request_duration_seconds_bucket  
      {job="inference"}[5m])  
  ))
```

Model Quality & Drift Dashboard



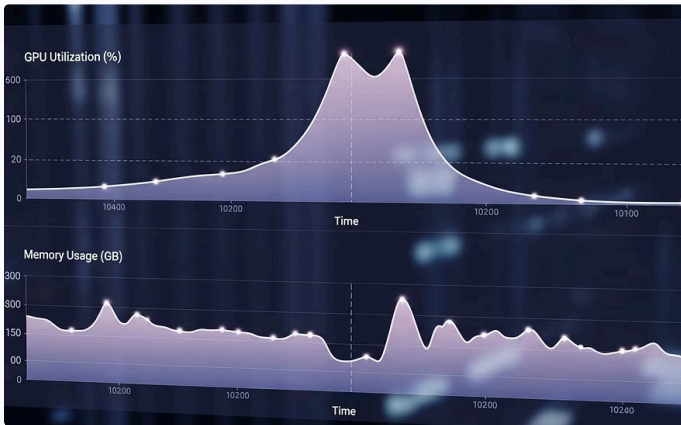
Essential Metrics

- Drift scores (PSI/KS/JS) over time
- Accuracy proxy (win-rate vs baseline)
- Prediction distribution histograms
- LLM-specific: refusal rate, toxicity %
- RAG hit-rate and relevance scores



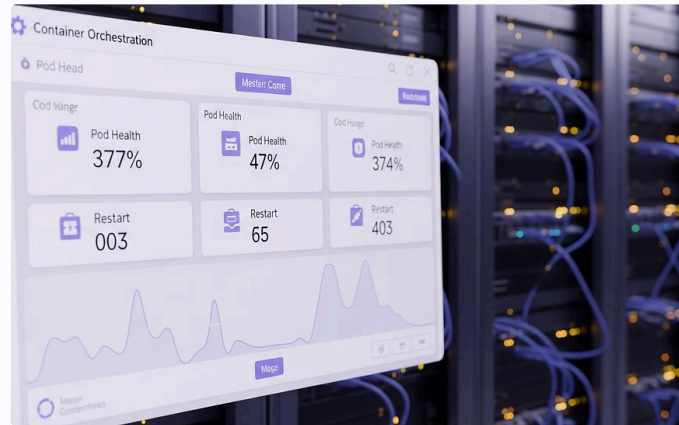
Set alerts when drift is sustained >24 hours

Infrastructure & Cluster Dashboard



Resource Utilization

GPU utilization %, memory %, power draw



Container Health

Pod restarts, CrashLoop counts, node pressure metrics



Quota & Resources

Quota usage per team/namespace with links to K8s events + logs

Data Freshness Dashboard

Critical Metrics

- Kafka consumer lag per group
- Airflow DAG runtime vs SLA
- Feature store staleness
- Per-feature freshness vs business SLO



Identify bottlenecks in your data pipelines before they affect model performance or business operations

Cost & Efficiency Dashboard

\$2.80

Per 1K Requests

Average inference cost

324

GPU Hours

This week's usage

22%

Idle Resources

Optimization potential

Key Cost Metrics

- GPU hours per team and environment
- Storage trend (hot vs cold tiers)
- Network egress by service
- Cost per training run



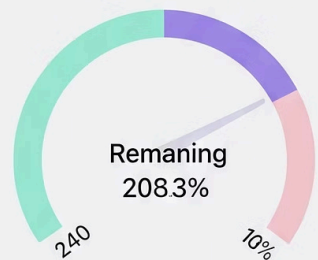
Alert Threshold

Set alerts to trigger on >20% daily spend spike to catch runaway costs early

SLO Compliance

Error Budget Consumption

Error Budget Consumption



Duck

Burn Rate



Dinck

Burn Rate Trend



SLO Burn Rate Dashboard

Error Budget Tracking

Monitor consumption rates against your service level objectives

- Fast burn alerts: 2% budget consumed in 1 hour
- Slow burn alerts: 2% budget consumed in 1 day

Visualization Components

Create separate dashboards for different SLO types:

- Latency SLO with gauges + trend lines
- Availability SLO with uptime tracking
- Drift SLO with baseline comparisons

Alerting Best Practices

Page on User Impact

High latency, 5xx errors, drift + KPI dip

Ticket for Background Issues

Storage growth, minor staleness, warnings

Link Everything

Connect alerts to dashboards, runbooks, and owners

Reduce Noise

Suppress duplicates and use hysteresis

```
- alert: HighP95Latency
  expr: histogram_quantile(0.95,
    sum by (le)(rate(
      http_request_duration_seconds_bucket[5m]
    ))) > 0.3
  for: 10m
  labels:
    severity: page
    team: platform
  annotations:
    summary: "p95 > 300ms"
    runbook: "https://runbooks/latency"
```

Stakeholder-Friendly Views

Executive Summary Dashboard

- Uptime percentage this week
- Cost trend vs budget
- Model drift trend (high-level)
- Business impact metrics

Use **sparklines + KPIs** instead of raw graphs



Translate technical metrics into business terms: "cost per transaction," "fraud detection uptime," "model confidence scores"

Adding Release Markers



Annotate Deploys

Push release markers on graphs



Compare Metrics

Before and after performance



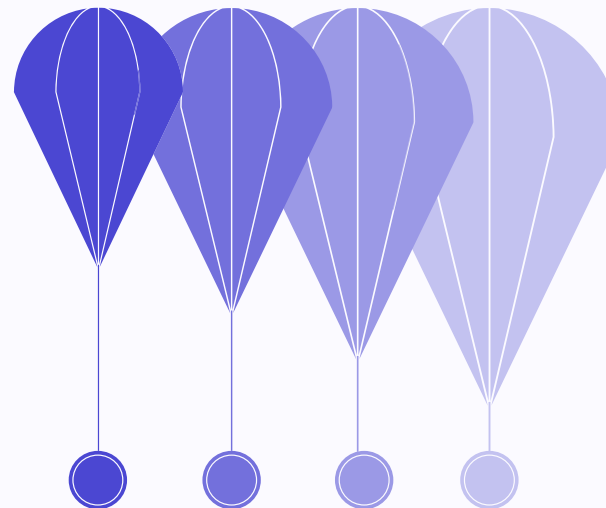
Canary Validation

Side-by-side canary panels



Evidence-Based Rollback

Decide with historical data



Implementation

- CI/CD pushes annotations on deploys
- Compare pre/post-release metrics
- Canary validation panels side by side

Benefits

- Makes rollback decisions evidence-based
- Provides historical context for issues
- Correlates performance changes with releases

Recommended Dashboard Layout



Top Row: SLO Gauges

Latency, error %, drift, and cost indicators



Middle: Detailed Time-Series

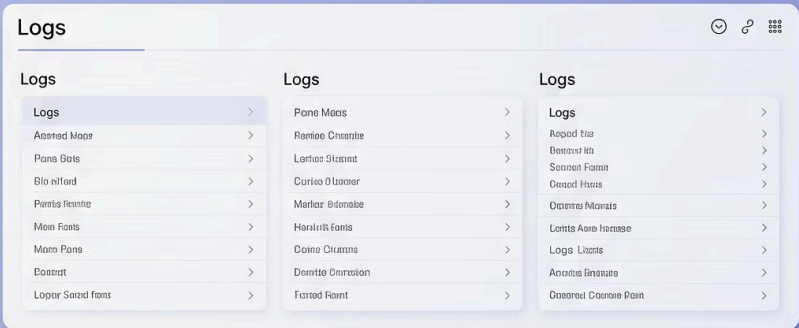
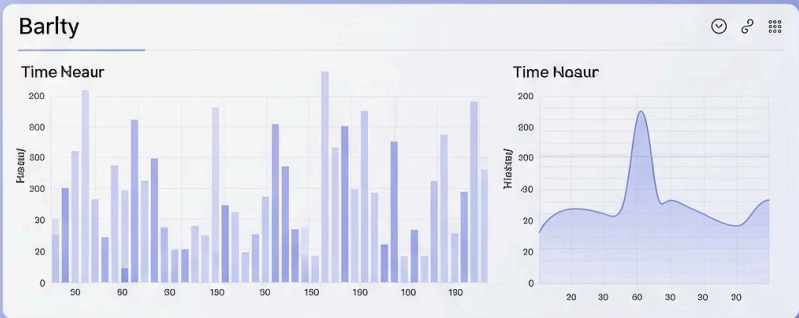
Latency histograms, drift trends, traffic patterns



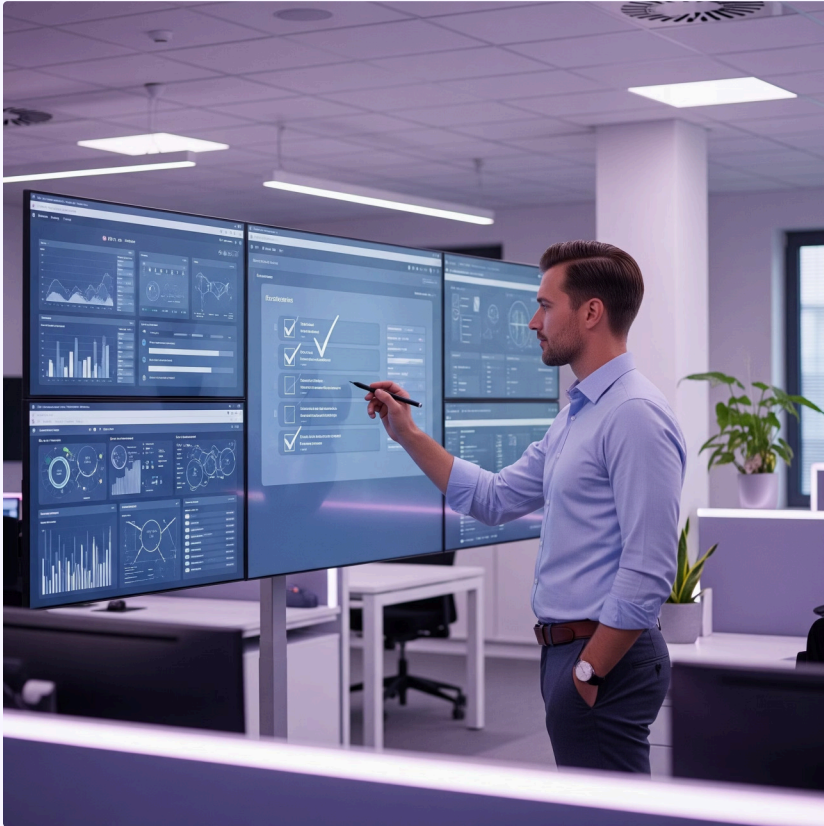
Bottom: Logs & Events

Related system events, error logs, and deployment markers

"Polished dashboards = clarity for both engineers & executives."



Capstone Checklist



- 5 core dashboards live (API, model, infra, data, cost)
- SLO gauges at top with clear thresholds
- Alerts linked to runbooks + owners
- Executive summary for stakeholders
- Release markers + drift overlays

✅ Present these in your Capstone to demonstrate production readiness

Summary

Polished Dashboards

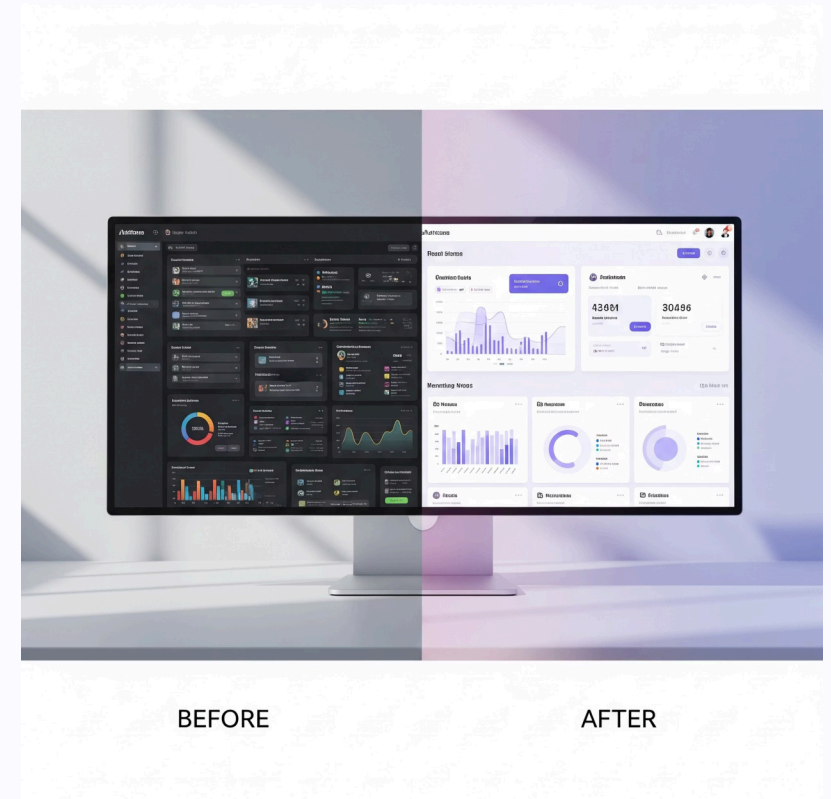
Elevate **signal over noise** to quickly identify issues

Align monitoring views to **personas & SLOs**

Make alerts **actionable & linked to runbooks**

Create **executive summaries** for non-technical reviewers

Your dashboards reflect the maturity of your ML system and observability practice





Discussion Prompt

Which dashboard will you prioritize polishing for your Capstone demo?

API Performance

Focus on latency, error rates, and request patterns

Model Drift

Emphasize data quality and prediction accuracy

Cost Visibility

Highlight efficiency and resource optimization